

Guide complet - Studio Booking Manager

Ce document explique l'architecture, le fonctionnement et le rôle de chaque fichier du projet.

1. Objectif

L'application permet de créer des cours, afficher les cours, inscrire un participant, afficher les cours complets, supprimer un cours (bonus) et quitter. Les données sont enregistrées dans SQLite grâce à SQLAlchemy ORM.

2. Architecture

main.py : point d'entrée, initialise la base, lance le menu et appelle les fonctions.

database.py : crée le moteur SQLAlchemy, les sessions et la classe Base.

models.py : définit la classe Course qui représente la table courses.

crud.py : contient toutes les opérations sur la base (CRUD).

menu.py : affiche le menu, les cours et les messages.

seed.py : ajoute des cours de démonstration si la base est vide.

3. Fonctionnement détaillé

Au lancement, Base.metadata.create_all() crée automatiquement les tables si elles n'existent pas. Ensuite seed() vérifie si la table est vide. Si oui, plusieurs cours sont ajoutés. La boucle while True affiche ensuite le menu jusqu'au choix Quitter.

4. SQLAlchemy ORM

L'ORM permet de manipuler des objets Python au lieu d'écrire directement du SQL. Les principales opérations sont : session.add() pour créer, session.query() pour lire, session.commit() pour enregistrer les modifications et session.delete() pour supprimer.

5. Classe Course

Les attributs id, name, coach, capacity, registered_count et price correspondent aux colonnes de la table. Les propriétés @property calculent automatiquement les places restantes, le chiffre d'affaires actuel, le chiffre d'affaires potentiel et indiquent si le cours est complet.

6. Déroulement d'une inscription

- 1) L'utilisateur choisit 'Inscrire'.
- 2) main.py demande l'identifiant du cours.
- 3) crud.py recherche le cours avec filter_by(id=...).

- 4) Si le cours n'existe pas, un message est retourné.
- 5) Si le cours est complet, l'inscription est refusée.
- 6) Sinon `registered_count` est incrémenté puis `session.commit()` enregistre la modification.

7. Pourquoi plusieurs fichiers ?

La séparation des responsabilités rend le code plus lisible, plus facile à maintenir et à tester. Chaque module possède un rôle unique.

8. Questions possibles en soutenance

Pourquoi utiliser un ORM ? Pour manipuler des objets Python au lieu d'écrire des requêtes SQL.

Pourquoi SQLite ? Base légère, locale et simple à déployer.

À quoi sert `commit()` ? À enregistrer définitivement les modifications.

Pourquoi utiliser `@property` ? Pour centraliser les calculs et éviter les répétitions.

Pourquoi une boucle `while True` ? Pour maintenir le menu jusqu'à la sortie.

Conclusion

Le projet met en œuvre la programmation orientée objet, SQLAlchemy ORM, SQLite et une architecture modulaire. Il répond au cahier des charges tout en restant évolutif.